

AVL TREE

Self-Balancing Binary Search Tree

Complete Implementation — 10 Programs in C++

- ✓ Q1 — Duplicate Value Handling
- ✓ Q2 — Non-Recursive Traversals (Stack-Based)
- ✓ Q3 — Reverse AVL (Left > Node > Right)
 - ✓ Q4 — Student Record Management
- ✓ Q5 — Terminal Node Display & Deletion
- ✓ Q6 — Inorder Successor & Predecessor
 - ✓ Q7 — Nodes at a Given Level
- ✓ Q8 — Display All Left-Child Nodes
- ✓ Q9 — Largest Value at a Given Level
- ✓ Q10 — Display All Right-Child Nodes

Files: AVL.h | AVL.cpp | main.cpp | Language: C++

Table of Contents

Table of Contents	2
1. Introduction to AVL Trees	4
1.1 Why AVL Over Plain BST?	4
1.2 AVL Balance Factor.....	4
1.3 The Four Rotation Cases.....	4
2. AVL Node Structure & Class Design (AVL.h)	6
2.1 AVLNode Structure.....	6
2.2 AVLTree Class — Complete Interface	6
2.3 ReverseAVL Class (Q3)	7
3. Core AVL Implementation (AVL.cpp)	8
3.1 height() and balanceFactor()	8
3.2 rotateRight() — Fixes Left-Left Imbalance	8
3.3 rotateLeft() — Fixes Right-Right Imbalance	8
3.4 balance() — The AVL Rebalancing Hub	8
3.5 Insert — Recursive with Rebalancing	9
3.6 Delete — Recursive with Rebalancing.....	9
4. The 10 Programs — Detailed Explanation	11
Q1 — Duplicate Value Handling	11
insertDup() — Allow Repeated Keys.....	11
deletionDup() — Remove One Occurrence at a Time	11
Q2 — Non-Recursive Traversals (Stack-Based).....	11
inorderNR() — Left → Root → Right.....	12
preorderNR() — Root → Left → Right	12
postorderNR() — Left → Right → Root (Two Stacks)	12
Q3 — Reverse AVL (Left > Node, Right < Node).....	13
Q4 — Student Record Management.....	13
Q5 — Terminal Node Display & Deletion.....	14
Q6 — Inorder Successor & Predecessor	15
Q7 — Nodes at a Given Level	16
Q8 — Display All Left-Child Nodes	16
Q9 — Largest Value at a Given Level	17
Q10 — Display All Right-Child Nodes.....	17
5. Complexity Analysis.....	18
5.1 AVL vs BST Complexity Comparison.....	18
5.2 All Operations in This Project	18

5.3 Space Used Per Node	18
6. Visual Reference — AVL Tree Diagrams	20
6.1 Building an AVL Tree Step by Step	20
6.2 Full Tree After Inserting 50,30,70,20,40,60,80	20
6.3 Reverse AVL Tree After Inserting 50,30,70,20,80	20
7. Pseudocode Summary	22
7.1 AVL Insert	22
7.2 AVL Delete	22
7.3 Non-Recursive Inorder	22
8. Key Concepts, Common Mistakes & Interview Tips	23
8.1 Key Points to Remember	23
8.2 Common Mistakes	23
8.3 Interview Cheat Sheet	23
8.4 Classic Interview Questions on AVL	24

1. Introduction to AVL Trees

An AVL tree (named after inventors Adelson-Velsky and Landis, 1962) is a self-balancing Binary Search Tree. It maintains the standard BST ordering property while additionally ensuring that the height difference (Balance Factor) between the left and right subtree of any node never exceeds 1.

1.1 Why AVL Over Plain BST?

A plain BST degenerates into a linked list when values are inserted in sorted order, making operations $O(n)$. An AVL tree prevents this by automatically rotating the tree after each insert or delete to keep the height at $O(\log n)$ always.

BST (unbalanced)	AVL Tree (always balanced)
$O(n)$ worst-case for insert/delete/search	$O(\log n)$ guaranteed for all operations
Can become a linked list	Height bounded by $1.44 * \log_2(n+2)$
No extra bookkeeping	Each node stores a height value
Simple rotations not required	Performs LL / LR / RR / RL rotations

1.2 AVL Balance Factor

```
Balance Factor (BF) of node N = Height(N.left) - Height(N.right)
```

```
Valid AVL state: BF ∈ { -1, 0, +1 }
```

```
BF = +2 => Left-heavy => Need Right or Left-Right rotation
```

```
BF = -2 => Right-heavy => Need Left or Right-Left rotation
```

1.3 The Four Rotation Cases

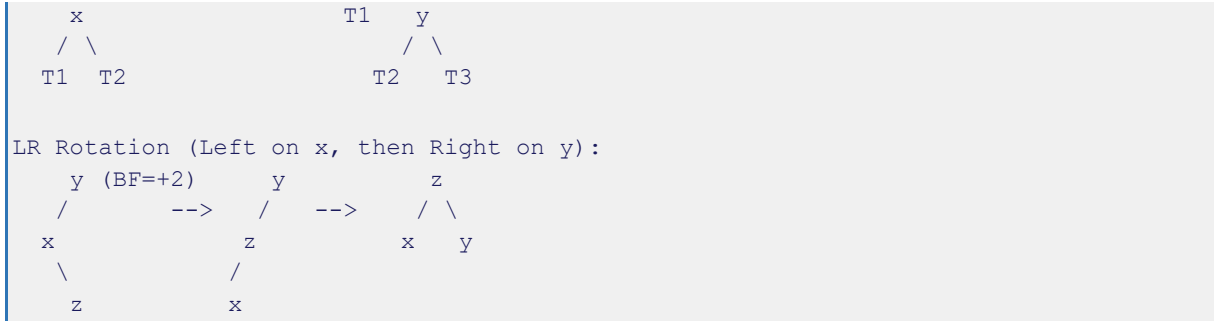
Case	Trigger	Fix
LL (Left-Left)	$BF > 1$ and left child $BF \geq 0$	Single Right Rotation
LR (Left-Right)	$BF > 1$ and left child $BF < 0$	Left Rot on child + Right Rot on node
RR (Right-Right)	$BF < -1$ and right child $BF \leq 0$	Single Left Rotation
RL (Right-Left)	$BF < -1$ and right child $BF > 0$	Right Rot on child + Left Rot on node

```
LL Rotation (Right Rotation on y):
```

```

      y (BF=+2)          x
     /             \
    /               --> / \

```



2. AVL Node Structure & Class Design (AVL.h)

2.1 AVLNode Structure

```
struct AVLNode {
    int      info;      // Key value (also Roll No. in Q4)
    int      count;     // Duplicate counter (Q1)
    int      height;    // AVL height – updated after every rotation
    string   name;      // Student name (Q4)
    float    marks;     // Student marks (Q4)
    AVLNode *lchild;    // Left child pointer
    AVLNode *rchild;    // Right child pointer
};
```

Why 'height' field?

Every AVL node stores its own height so that height() is $O(1)$.

Without storing height, computing balance factor would require $O(n)$ traversal each time.

height is updated immediately after every rotation in rotateRight() and rotateLeft().

A NULL node has height 0 by convention.

2.2 AVLTree Class — Complete Interface

```
class AVLTree {
public:
    AVLNode *root;

    // CORE AVL HELPERS
    int      height(AVLNode *n);
    int      balanceFactor(AVLNode *n);
    AVLNode* rotateRight(AVLNode *y);
    AVLNode* rotateLeft(AVLNode *x);
    AVLNode* balance(AVLNode *n);

    // Q1: Standard + Duplicate-aware insert/delete
    AVLNode* insertNode(AVLNode *node, int item);
    AVLNode* deleteNode(AVLNode *node, int item);
    AVLNode* minValueNode(AVLNode *node);
    void insert(int item);      void deletion(int item);
    void insertDup(int item);   void deletionDup(int item);

    // Q2: Recursive + Non-recursive traversals
    void inorder(AVLNode *ptr); void preorder(AVLNode *ptr);
    void postorder(AVLNode *ptr);
    void inorderNR();   void preorderNR();   void postorderNR();

    // Q4: Student records
    void insertStudent(int rollNo, const string &name, float marks);
    void searchStudent(int rollNo);
};
```

```
// Q5-Q10: Advanced features
void displayLeaves(AVLNode *ptr);
void deleteLeaves();
void findSuccessorPredecessor(int item);
void nodesAtLevel(AVLNode *ptr, int target, int current);
void displayLeftNodes(AVLNode *ptr);
int largestAtLevel(AVLNode *ptr, int target, int current);
void displayRightNodes(AVLNode *ptr);
};
```

2.3 ReverseAVL Class (Q3)

A completely separate class with its own RevAVLNode struct. This ensures the main AVLTree class remains untouched. ReverseAVL uses a reversed insertion rule (larger → left, smaller → right) but applies the same AVL rotations to maintain balance.

3. Core AVL Implementation (AVL.cpp)

3.1 height() and balanceFactor()

```
int AVLTree::height(AVLNode *n) {
    return (n == NULL) ? 0 : n->height; // NULL has height 0
}

int AVLTree::balanceFactor(AVLNode *n) {
    return (n == NULL) ? 0 : height(n->lchild) - height(n->rchild);
}
```

3.2 rotateRight() — Fixes Left-Left Imbalance

```
AVLNode* AVLTree::rotateRight(AVLNode *y) {
    AVLNode *x = y->lchild; // x becomes new root of subtree
    AVLNode *T2 = x->rchild; // T2 will move to y's left

    x->rchild = y; // y goes under x
    y->lchild = T2; // T2 re-attached

    // Update heights - y first because it is now LOWER
    y->height = 1 + MAX(height(y->lchild), height(y->rchild));
    x->height = 1 + MAX(height(x->lchild), height(x->rchild));
    return x;
}
```

3.3 rotateLeft() — Fixes Right-Right Imbalance

```
AVLNode* AVLTree::rotateLeft(AVLNode *x) {
    AVLNode *y = x->rchild;
    AVLNode *T2 = y->lchild;

    y->lchild = x;
    x->rchild = T2;

    x->height = 1 + MAX(height(x->lchild), height(x->rchild));
    y->height = 1 + MAX(height(y->lchild), height(y->rchild));
    return y;
}
```

3.4 balance() — The AVL Rebalancing Hub

Called after every insert and delete on the way back up the recursion. It checks the balance factor and applies the correct rotation.

```
AVLNode* AVLTree::balance(AVLNode *n) {
    if (n == NULL) return NULL;
```



```
n->height = 1 + MAX(height(n->lchild), height(n->rchild));
int bf = balanceFactor(n);

if (bf > 1 && balanceFactor(n->lchild) >= 0)    // LL
    return rotateRight(n);
if (bf > 1 && balanceFactor(n->lchild) < 0) {    // LR
    n->lchild = rotateLeft(n->lchild);
    return rotateRight(n);
}
if (bf < -1 && balanceFactor(n->rchild) <= 0)    // RR
    return rotateLeft(n);
if (bf < -1 && balanceFactor(n->rchild) > 0) {    // RL
    n->rchild = rotateRight(n->rchild);
    return rotateLeft(n);
}
return n;    // Already balanced
}
```

3.5 Insert — Recursive with Rebalancing

```
AVLNode* AVLTree::insertNode(AVLNode *node, int item) {
    // Step 1: Standard BST insertion
    if (node == NULL) {
        AVLNode *tmp = new AVLNode();
        tmp->info = item; tmp->count = 1; tmp->height = 1;
        tmp->lchild = tmp->rchild = NULL;
        return tmp;
    }
    if (item < node->info)
        node->lchild = insertNode(node->lchild, item);
    else if (item > node->info)
        node->rchild = insertNode(node->rchild, item);
    else { cout << 'Item already present'; return node; }

    // Step 2: Rebalance on the way back up
    return balance(node);
}

void AVLTree::insert(int item) { root = insertNode(root, item); }
```

3.6 Delete — Recursive with Rebalancing

```
AVLNode* AVLTree::deleteNode(AVLNode *node, int item) {
    if (node == NULL) { cout << 'Not found'; return NULL; }
    if (item < node->info)
        node->lchild = deleteNode(node->lchild, item);
    else if (item > node->info)
        node->rchild = deleteNode(node->rchild, item);
    else { // Found
        if (node->lchild == NULL || node->rchild == NULL) {
            AVLNode *tmp = node->lchild ? node->lchild : node->rchild;
            if (tmp == NULL) { delete node; return NULL; } // Leaf
```

```
        else { *node = *tmp; delete tmp; }           // 1 child
    } else {
        // 2 children: replace with inorder successor
        AVLNode *suc = minValueNode(node->rchild);
        node->info    = suc->info;
        node->rchild = deleteNode(node->rchild, suc->info);
    }
}
return balance(node); // Rebalance on the way back up
}
```

Key difference from BST delete

In BST delete, we physically splice out a node using parent pointers.

In AVL delete, we use RECURSION and return the updated subtree root.

balance() is called on every ancestor on the way back up automatically.

This elegantly handles all rebalancing without explicit parent tracking.

4. The 10 Programs — Detailed Explanation

Q1 — Duplicate Value Handling

Concept: Standard AVL rejects duplicate keys. insertDup() allows the same key to be inserted multiple times by incrementing a 'count' field instead of creating a new node. deletionDup() decrements count before physically removing the node.

insertDup() — Allow Repeated Keys

```
AVLNode* AVLTree::insertDupNode(AVLNode *node, int item) {
    if (node == NULL) { /* create new node with count=1 */ }
    if (item < node->info)
        node->lchild = insertDupNode(node->lchild, item);
    else if (item > node->info)
        node->rchild = insertDupNode(node->rchild, item);
    else {
        node->count++; // KEY: increment count, no new node
        return node;  // No structural change: no rebalance needed
    }
    return balance(node);
}
```

deletionDup() — Remove One Occurrence at a Time

```
AVLNode* AVLTree::deleteDupNode(AVLNode *node, int item) {
    // ... navigate to node ...
    else { // Found
        if (node->count > 1) {
            node->count--; // Just decrement — do NOT remove node
            return node;  // No structural change
        }
        // count == 1: remove physically (same as deleteNode)
        // ... case A/B/C logic ...
    }
    return balance(node);
}
```

Example Trace

```
insertDup(50) → node created, count=1
insertDup(50) → count=2 (no new node, no rotation)
insertDup(50) → count=3
deletionDup(50) → count=2
deletionDup(50) → count=1
deletionDup(50) → count==1, node physically removed + rebalance
```

Q2 — Non-Recursive Traversals (Stack-Based)

These three traversals use std::stack explicitly, mimicking what the CPU call stack does implicitly in recursive versions. They produce identical output to the recursive versions.

inorderNR() — Left → Root → Right

```
void AVLTree::inorderNR() {
    stack<AVLNode *> stk;
    AVLNode *ptr = root;
    while (ptr != NULL || !stk.empty()) {
        while (ptr != NULL) { // Phase 1: go as far left as possible
            stk.push(ptr);
            ptr = ptr->lchild;
        }
        ptr = stk.top(); stk.pop(); // Phase 2: visit
        cout << ptr->info << ' ';
        ptr = ptr->rchild; // Phase 3: try right subtree
    }
}
```

preorderNR() — Root → Left → Right

```
void AVLTree::preorderNR() {
    stack<AVLNode *> stk;
    stk.push(root);
    while (!stk.empty()) {
        AVLNode *ptr = stk.top(); stk.pop();
        cout << ptr->info << ' '; // Visit before children
        if (ptr->rchild) stk.push(ptr->rchild); // Right pushed first
        if (ptr->lchild) stk.push(ptr->lchild); // Left processed first
    }
}
```

postorderNR() — Left → Right → Root (Two Stacks)

```
void AVLTree::postorderNR() {
    stack<AVLNode *> stk1, stk2;
    stk1.push(root);
    while (!stk1.empty()) {
        AVLNode *ptr = stk1.top(); stk1.pop();
        stk2.push(ptr); // Collect in reverse order
        if (ptr->lchild) stk1.push(ptr->lchild);
        if (ptr->rchild) stk1.push(ptr->rchild);
    }
    while (!stk2.empty()) { // Print in postorder
        cout << stk2.top()->info << ' ';
        stk2.pop();
    }
}
```

Stack Trace — inorderNR on {50,30,70,20,40}

ptr=50 → push 50, go left
ptr=30 → push 30, go left
ptr=20 → push 20, go left → NULL
Pop 20, print 20, ptr=NULL (no right child)
Pop 30, print 30, ptr=40
ptr=40 → push 40, go left → NULL
Pop 40, print 40, ptr=NULL

Pop 50, print 50, ptr=70 ...
Output: 20 30 40 50 70

Q3 — Reverse AVL (Left > Node, Right < Node)

Implemented as a separate ReverseAVL class. The ordering is inverted: larger values go to the LEFT subtree and smaller values go to the RIGHT subtree. The AVL balancing rotations remain exactly the same — only the comparison direction changes.

```
RevAVLNode* ReverseAVL::insertRev(RavAVLNode *node, int item) {
    if (node == NULL) { /* create node */ }
    if (item == node->info) { /* reject duplicate */ return node; }

    // REVERSED RULE: larger → LEFT, smaller → RIGHT
    if (item > node->info)
        node->lchild = insertRev(node->lchild, item);
    else
        node->rchild = insertRev(node->rchild, item);

    node->height = 1 + MAX(height(node->lchild), height(node->rchild));
    int bf = balanceFactor(node);
    // Same 4-case rotation logic as normal AVL ...
    return node;
}
```

Example: Insert 50, 30, 70, 20, 80 into ReverseAVL

70 > 50 → goes LEFT (not right!)

30 < 50 → goes RIGHT (not left!)

Result:

```

      50
     /\
    70 30 (70 left, 30 right)
   /\
  80 20
```

Inorder traversal output: 80 70 50 30 20 (Descending!)

Q4 — Student Record Management

The AVLNode already contains 'name' (string) and 'marks' (float) fields. insertStudent() uses Roll No as the BST key and stores name and marks as satellite data. searchStudent() uses the existing searchNode() recursive function.

```
AVLNode* AVLTree::insertStudentNode(AVLNode *node, int rollNo,
                                   const string &name, float marks) {
    if (node == NULL) {
        AVLNode *tmp = new AVLNode();
        tmp->info = rollNo; // Roll No is the BST key
```

```
        tmp->name = name;    // Satellite data
        tmp->marks = marks;  // Satellite data
        tmp->height = 1; tmp->count = 1;
        tmp->lchild = tmp->rchild = NULL;
        return tmp;
    }
    if (rollNo < node->info)
        node->lchild = insertStudentNode(node->lchild, rollNo, name, marks);
    else if (rollNo > node->info)
        node->rchild = insertStudentNode(node->rchild, rollNo, name, marks);
    else { cout << 'Already present'; return node; }
    return balance(node);
}

void AVLTree::searchStudent(int rollNo) {
    AVLNode *ptr = searchNode(root, rollNo); // Recursive BST search
    if (ptr == NULL) { cout << 'Not found'; return; }
    cout << 'Roll No: ' << ptr->info << endl;
    cout << 'Name   : ' << ptr->name << endl;
    cout << 'Marks  : ' << ptr->marks << endl;
}
```

Example

insertStudent(101, 'Alice', 92.5)

insertStudent(103, 'Bob', 78.0)

insertStudent(102, 'Carol', 88.5)

AVL tree ordered by Roll No: 101→102→103 (auto-balanced)

searchStudent(102):

Roll No: 102 Name: Carol Marks: 88.5

Q5 — Terminal Node Display & Deletion

A terminal (leaf) node has both lchild == NULL and rchild == NULL. displayLeaves() recursively traverses and prints them. deleteLeavesHelper() removes all leaves while calling balance() on each ancestor to maintain AVL property.

```
void AVLTree::displayLeaves(AVLNode *ptr) {
    if (ptr != NULL) {
        if (ptr->lchild == NULL && ptr->rchild == NULL)
            cout << ptr->info << ' '; // It's a leaf
        else {
            displayLeaves(ptr->lchild);
            displayLeaves(ptr->rchild);
        }
    }
}

AVLNode* AVLTree::deleteLeavesHelper(AVLNode *ptr) {
    if (ptr == NULL) return NULL;
    if (ptr->lchild == NULL && ptr->rchild == NULL) {
```

```

        delete ptr;
        return NULL;    // This node is now gone
    }
    ptr->lchild = deleteLeavesHelper(ptr->lchild);
    ptr->rchild = deleteLeavesHelper(ptr->rchild);
    return balance(ptr); // Rebalance after structural change
}

```

Example on {50,30,70,20,40,60,80}

Leaves: 20, 40, 60, 80

After deleteLeaves():

```

    50
   / \
  30  70

```

(30 and 70 are now leaves — 20,40,60,80 are gone)

Inorder after: 30 50 70

Q6 — Inorder Successor & Predecessor

Inorder Predecessor: The largest value smaller than the given key — the rightmost node in the left subtree, or the last left-turn ancestor.

Inorder Successor: The smallest value larger than the given key — the leftmost node in the right subtree, or the last right-turn ancestor.

```

void AVLTree::findSuccessorPredecessor(int item) {
    AVLNode *ptr=root, *successor=NULL, *predecessor=NULL;
    while (ptr != NULL) {
        if (item == ptr->info) {
            // Predecessor = rightmost in left subtree
            if (ptr->lchild) {
                AVLNode *t = ptr->lchild;
                while (t->rchild) t = t->rchild;
                predecessor = t;
            }
            // Successor = leftmost in right subtree
            if (ptr->rchild) {
                AVLNode *t = ptr->rchild;
                while (t->lchild) t = t->lchild;
                successor = t;
            }
            break;
        } else if (item < ptr->info) {
            successor = ptr; ptr = ptr->lchild;
        } else {
            predecessor = ptr; ptr = ptr->rchild;
        }
    }
}

```

Example: findSuccessorPredecessor(50) in {20,30,40,50,60,70,80}

Found node 50 (the root)
Left subtree of 50: {30,20,40}
Rightmost of left subtree = 40 → Predecessor = 40
Right subtree of 50: {70,60,80}
Leftmost of right subtree = 60 → Successor = 60

Output:
Inorder Predecessor of 50 : 40
Inorder Successor of 50 : 60

Q7 — Nodes at a Given Level

Root is at level 1. The function recursively decrements the target level as it descends; when current equals target, the node is printed.

```
void AVLTree::nodesAtLevel(AVLNode *ptr, int target, int current) {
    if (ptr == NULL) return;
    if (current == target) {
        cout << ptr->info << ' '; // Reached target level
        return;                    // No need to go deeper
    }
    nodesAtLevel(ptr->lchild, target, current + 1);
    nodesAtLevel(ptr->rchild, target, current + 1);
}
// Usage: tree.nodesAtLevel(tree.root, level, 1);
```

Example: Level 2 of {50,30,70,20,40,60,80}

Call: nodesAtLevel(root=50, target=2, current=1)
current(1) != target(2) → recurse
nodesAtLevel(30, 2, 2): current==target → PRINT 30
nodesAtLevel(70, 2, 2): current==target → PRINT 70
Output: 30 70

Level 3 output: 20 40 60 80

Q8 — Display All Left-Child Nodes

Every node that is a left child of its parent is printed. The function walks the tree recursively, and wherever a node has a left child, that child's info is printed.

```
void AVLTree::displayLeftNodes(AVLNode *ptr) {
    if (ptr == NULL) return;
    if (ptr->lchild != NULL) {
        cout << ptr->lchild->info << ' '; // Print the left child
        displayLeftNodes(ptr->lchild);    // Recurse into it
    }
    displayLeftNodes(ptr->rchild); // Also check right subtree
}
```


Example on {50,30,70,20,40,60,80}

At 50: lchild=30 → print 30

At 30: lchild=20 → print 20

At 20: lchild=NULL → skip

At 70: lchild=60 → print 60

At 60: lchild=NULL → skip

Output: 30 20 60

Q9 — Largest Value at a Given Level

Recursively visits all nodes at the target level and returns the maximum value found. Returns INT_MIN if no nodes exist at that level (used to detect an empty level).

```
int AVLTree::largestAtLevel(AVLNode *ptr, int target, int current) {
    if (ptr == NULL) return INT_MIN; // No node: return minimum sentinel
    if (current == target) return ptr->info; // At level: return value
    int leftMax = largestAtLevel(ptr->lchild, target, current+1);
    int rightMax = largestAtLevel(ptr->rchild, target, current+1);
    return (leftMax > rightMax) ? leftMax : rightMax;
}
```

Example: largestAtLevel(root,2,1) on {50,30,70}

At 50 (current=1): recurse

Left: ptr=30, current=2==target → return 30

Right: ptr=70, current=2==target → return 70

leftMax=30, rightMax=70 → return 70

Output: Largest value at level 2: 70

Q10 — Display All Right-Child Nodes

Mirror image of Q8 — prints every node that serves as a right child of its parent.

```
void AVLTree::displayRightNodes(AVLNode *ptr) {
    if (ptr == NULL) return;
    if (ptr->rchild != NULL) {
        cout << ptr->rchild->info << ' '; // Print the right child
        displayRightNodes(ptr->rchild); // Recurse into it
    }
    displayRightNodes(ptr->lchild); // Also check left subtree
}
```

Example on {50,30,70,20,40,60,80}

At 50: rchild=70 → print 70

At 70: rchild=80 → print 80

At 30: rchild=40 → print 40

Output: 70 80 40

5. Complexity Analysis

5.1 AVL vs BST Complexity Comparison

Operation	BST (avg / worst)	AVL (always)
insert	$O(\log n)$ / $O(n)$	$O(\log n)$
delete	$O(\log n)$ / $O(n)$	$O(\log n)$
search	$O(\log n)$ / $O(n)$	$O(\log n)$
Rotations per insert	0	At most 2 (1 double or 2 single)
Rotations per delete	0	$O(\log n)$ rotations possible
Height	$O(n)$ worst	$\leq 1.44 * \log_2(n+2)$

5.2 All Operations in This Project

Function	Time	Space	Notes
insertNode / deleteNode	$O(\log n)$	$O(\log n)$ stack	Recursive — stack depth = height
insertDup / deletionDup	$O(\log n)$	$O(\log n)$ stack	Same as insert/delete
inorder / preorder / postorder (rec)	$O(n)$	$O(\log n)$ stack	Visits all nodes
inorderNR / preorderNR	$O(n)$	$O(\log n)$ explicit stack	Stack holds at most h nodes
postorderNR	$O(n)$	$O(n)$	stk2 holds all n nodes
insertStudent / searchStudent	$O(\log n)$	$O(\log n)$	Same tree structure
displayLeaves / deleteLeaves	$O(n)$	$O(\log n)$	Must check all nodes
findSuccessorPredecessor	$O(\log n)$	$O(1)$	Single path traversal
nodesAtLevel	$O(n)$	$O(\log n)$	May visit all nodes
largestAtLevel	$O(n)$	$O(\log n)$	Checks all nodes at level
displayLeftNodes / displayRightNodes	$O(n)$	$O(\log n)$	Full traversal
ReverseAVL insert	$O(\log n)$	$O(\log n)$	Same AVL rotations

5.3 Space Used Per Node

```
sizeof(AVLNode) breakdown:
    int  info    = 4  bytes
    int  count   = 4  bytes
```

```
int height = 4 bytes    <- extra field vs plain BST
string name = ~32 bytes  (SSO + heap, platform-dependent)
float marks = 4 bytes
pointer lchild = 8 bytes (64-bit)
pointer rchild = 8 bytes
Total: ~64 bytes per node (integer-only mode: ~28 bytes)
```

6. Visual Reference — AVL Tree Diagrams

6.1 Building an AVL Tree Step by Step

```
Insert: 10, 20, 30 (triggers RR rotation)

Step 1: Insert 10      Step 2: Insert 20      Step 3: Insert 30
  10                    10                    10 (BF=-2, RR!)
                      \                      \
                      20                      20
                                           \
                                           30

After RR (rotateLeft on 10):
    20
   /  \
  10   30  <- Balanced! BF=0 everywhere
```

```
Insert: 30, 20, 10 (triggers LL rotation)

    30 (BF=+2)  ->  rotateRight(30)  ->    20
   /                                     /  \
  20                                 10   30
 /
10
```

```
Insert: 30, 10, 20 (triggers LR rotation)

    30 (BF=+2)  Step1: rotateLeft(10)  Step2: rotateRight(30)
   /
  10 (BF=-1)  ->    30                  ->    20
   \            / \                    /  \
  20           20  10                 10   30
               /
              10
```

6.2 Full Tree After Inserting 50,30,70,20,40,60,80

```

      50          BF=0  Height=3
     /  \
    30   70      BF=0  Height=2
   /  \  /  \
  20  40 60  80  BF=0  Height=1 (leaves)

All balance factors = 0. Perfect AVL tree.
Height = 3 = floor(log2(7)) + 1 [optimal]
```

6.3 Reverse AVL Tree After Inserting 50,30,70,20,80

```
Normal AVL:          Reverse AVL:
      50              50
     /  \            /  \
    30   70          70   30 (reversed!)
   /     \          /     \
  20      80        80      20

Inorder Normal:  20 30 50 70 80 (ascending)
Inorder Reverse: 80 70 50 30 20 (descending)
```

7. Pseudocode Summary

7.1 AVL Insert

```
AVL_INSERT(node, item):
    IF node == NULL THEN return new_node(item, height=1)
    IF item < node.info THEN node.left = AVL_INSERT(node.left, item)
    IF item > node.info THEN node.right = AVL_INSERT(node.right, item)
    ELSE PRINT 'duplicate'; return node
    node.height = 1 + MAX(H(node.left), H(node.right))
    bf = H(node.left) - H(node.right)
    IF bf > 1 AND bf(node.left) >= 0 THEN return ROTATE_RIGHT(node) [LL]
    IF bf > 1 AND bf(node.left) < 0 THEN node.left = ROTATE_LEFT(node.left);
    return ROTATE_RIGHT(node) [LR]
    IF bf < -1 AND bf(node.right) <= 0 THEN return ROTATE_LEFT(node) [RR]
    IF bf < -1 AND bf(node.right) > 0 THEN node.right = ROTATE_RIGHT(node.right);
    return ROTATE_LEFT(node) [RL]
    return node
```

7.2 AVL Delete

```
AVL_DELETE(node, item):
    IF node == NULL THEN return NULL (not found)
    IF item < node.info THEN node.left = AVL_DELETE(node.left, item)
    IF item > node.info THEN node.right = AVL_DELETE(node.right, item)
    ELSE // Found
        IF node has 0 or 1 child THEN
            tmp = the one child (or NULL for leaf)
            IF tmp == NULL THEN delete node; return NULL
            ELSE copy tmp into node; delete tmp
        ELSE // 2 children
            suc = leftmost(node.right) // inorder successor
            node.info = suc.info
            node.right = AVL_DELETE(node.right, suc.info)
    return BALANCE(node)
```

7.3 Non-Recursive Inorder

```
INORDER_NR(root):
    stack S; ptr = root
    WHILE ptr != NULL OR S not empty:
        WHILE ptr != NULL: PUSH(S, ptr); ptr = ptr.left
        ptr = POP(S); PRINT ptr.info; ptr = ptr.right
```

8. Key Concepts, Common Mistakes & Interview Tips

8.1 Key Points to Remember

- **AVL invariant:** Balance Factor = $\text{height}(\text{left}) - \text{height}(\text{right})$ must be -1, 0, or +1 for EVERY node.
- **Height update order:** Always update height of the LOWER node first during rotation, then the upper node.
- **balance() is called bottom-up:** After each recursive insert/delete call returns, balance() is applied on the way back up.
- **Inorder successor in delete:** The leftmost node in the right subtree — has at most one child (right), so removal is simple.
- **Duplicate handling:** incrementing count does NOT change tree structure, so no rebalance is needed for duplicates.
- **Reverse AVL:** Only the comparison direction changes. Rotations are identical to normal AVL.
- **deleteLeaves calls balance():** Because removing leaves changes subtree heights and may upset balance factors of ancestors.

8.2 Common Mistakes

- Forgetting to update height before computing balance factor in balance().
- Updating the parent node's height before the child's after a rotation.
- Using malloc instead of new for AVLNode (string member needs constructor).
- Not returning the new subtree root from insertNode/deleteNode (the root may change after rotation).
- Confusing LL/LR cases: check the balance factor of the CHILD, not just the node.
- In deleteNode case-B (one child), do `*node = *tmp` then `delete tmp` — not the other way around.

8.3 Interview Cheat Sheet

Quick Reference

AVL guarantees $O(\log n)$ — BST worst case is $O(n)$
Rotations per insert: at most 1 (single or double)
Rotations per delete: up to $O(\log n)$
Height of AVL(n): at most $1.44 * \log_2(n+2) - 0.328$
Inorder gives sorted output (ascending)
Reverse AVL inorder gives descending output
4 rotation cases: LL, LR, RR, RL
balance() must be called on EVERY ancestor after insert/delete
Always use 'new' not 'malloc' when struct contains std::string

8.4 Classic Interview Questions on AVL

1. What is the difference between a BST and an AVL tree?
2. What is the maximum height of an AVL tree with n nodes?
3. How many rotations occur in the worst case during insertion?
4. Why do we use the inorder successor (not predecessor) in deletion?
5. Can an AVL tree have a balance factor of 2 after a single insertion?
6. What happens if we insert already-sorted values into a BST vs AVL?
7. Explain the LR (Left-Right) rotation with an example.
8. Why is deleteLeaves followed by balance() in the AVL implementation?

